

Deep Learning

Introdução a Redes Convolucionais

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

maio de 2026

Overview

1. **Motivação**
2. **Invariância e Equivariância**
3. **Convolução**
4. **Camada Convolutacional**
5. **Campos Receptivos**
6. **CNN em 2D**
7. **Treinamento**
8. **Downsampling e Upsampling**
9. **Implementação de Convolução**
10. **Referências**

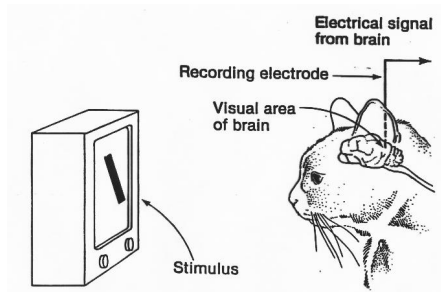
Overview

1. **Motivação**
2. Invariância e Equivariância
3. Convolução
4. Camada Convolutiva
5. Campos Receptivos
6. CNN em 2D
7. Treinamento
8. Downsampling e Upsampling
9. Implementação de Convolução
10. Referências

Motivação Histórica

- David Hubel e Torsten Wiesel realizaram experimentos implantando eletrodos no córtex visual de gatos^a.
- Aparentemente o processamento começa identificando padrões simples, como arestas.
 - Vídeo
- Camadas seguintes combinam esses padrões em estruturas mais complexas.

^aDavid H. Hubel e Torsten N. Wiesel. "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex". Em: The Journal of Physiology 160.1 (1962), pp. 106–154.



Redes Convolucionais

- São redes que incluem operações de convolução.
 - Usadas principalmente em imagens.
- Criadas em 1989 por Yann LeCun, com a LeNet 5¹.
- Explodiram em popularidade após 2012, com a AlexNet².
 - Treinamento em GPUs.
 - Mais dados.
 - Mais poder computacional.

¹Yann LeCun et al. “Handwritten digit recognition with a back-propagation network”. Em: Advances in Neural Information Processing Systems 2 (1989).

²Alex Krizhevsky, Ilya Sutskever e Geoffrey E. Hinton. “ImageNet classification with deep convolutional neural networks”. Em: Advances in Neural Information Processing Systems. Vol. 25. 2012.

Problemas com MLP

Imagens têm três propriedades que sugerem o uso de arquiteturas específicas:

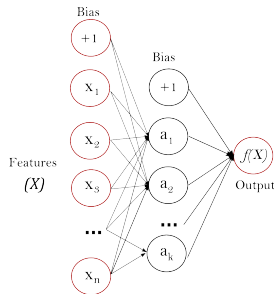
1. Alta dimensionalidade.

- Uma imagem 512×512 resulta em 262,144 pixels.
- Com 100 unidades na primeira camada oculta de um MLP teríamos 26,214,500 pesos.
- Considerando `float32`, são cerca de 800 MB só para a primeira camada.

2. Alta coesão espacial.

- Pixels próximos têm valores altamente correlacionados.
- Uma característica (orelha de um gato) pode aparecer em vários locais distintos.

3. Semântica estável frente a transformações geométricas



Camadas Convolucionais

Camadas convolucionais trazem diversos benefícios:

- Processam regiões da imagem separadamente, compartilhando pesos.
- Usam menos parâmetros do que camadas totalmente conectadas.
- Exploram a correlação entre pixels próximos.
- Não precisam reaprender padrões em locais diferentes da imagem.

Overview

1. Motivação
- 2. Invariância e Equivariância**
3. Convolução
4. Camada Convolutiva
5. Campos Receptivos
6. CNN em 2D
7. Treinamento
8. Downsampling e Upsampling
9. Implementação de Convolução
10. Referências

Invariância

- Uma função f aplicada a uma imagem x é dita **invariante** a uma transformação t se:

$$f[t[x]] = f[x]$$

- Ou seja, independente da transformação aplicada na entrada, o resultado é o mesmo.
- Exemplo: classificar a imagem como “gato” não deveria depender de o gato estar à esquerda ou à direita do quadro.

Equivariância

- Uma função f aplicada a uma imagem x é dita **equivariante** a uma transformação t se:

$$f[t[x]] = t[f[x]]$$

- Ou seja, aplicar a transformação na entrada equivale a aplicá-la na saída.
- Exemplo: transladar a imagem de entrada deve resultar na mesma segmentação transladada.



Overview

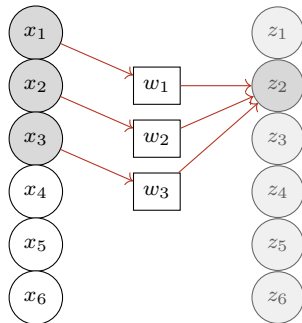
1. Motivação
2. Invariância e Equivariância
- 3. Convolução**
4. Camada Convolutacional
5. Campos Receptivos
6. CNN em 2D
7. Treinamento
8. Downsampling e Upsampling
9. Implementação de Convolução
10. Referências

Convolução 1D

- A convolução transforma um vetor de entradas x em um vetor de saídas z , tal que:

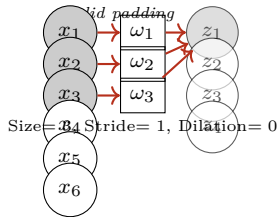
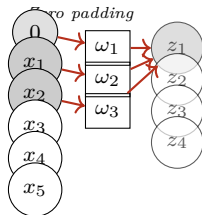
$$z_i = w_1 x_{i-1} + w_2 x_i + w_3 x_{i+1}$$

- Os pesos são **compartilhados** por toda a entrada.
- Esse conjunto de pesos é chamado de *kernel* ou *filtro*.



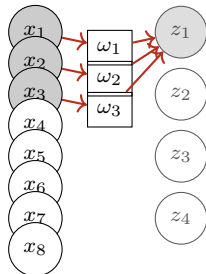
Padding

- Como lidar com os extremos da entrada?
 - *Zero padding*: completar a entrada com zeros.
 - *Valid padding*: aplicar a convolução apenas onde o *kernel* cabe inteiramente.
 - Entrada “circular” ou periódica (pouco utilizado).



Stride

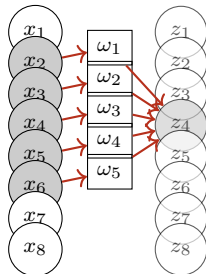
- *Stride* é o deslocamento do *kernel* a cada passo.
- Pode ser interpretado como *downsampling* quando maior que 1.
- Exemplo: Size= 3, Stride= 2 produz uma saída com aproximadamente metade do comprimento da entrada.



Size= 3, Stride= 2, Dilation= 0

Kernel Size

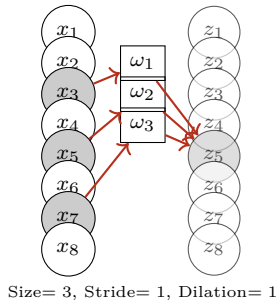
- *Kernel size* é o tamanho do filtro.
- Permite integrar informações espaciais mais distantes.
 - Mas aumenta o número de pesos a aprender.
- Exemplo: Size= 5, Stride= 1 olha 5 entradas vizinhas para produzir cada saída.



Size= 5, Stride= 1, Dilation= 0

Dilation

- *Dilation* integra informações mais distantes **sem** aumento do número de parâmetros.
 - Equivalente a aumentar o *kernel* deixando alguns pesos zerados.
- Exemplo: Size= 3, Stride= 1, Dilation= 1 olha posições $i-2, i, i+2$ em vez de $i-1, i, i+1$.



Overview

1. Motivação
2. Invariância e Equivariância
3. Convolução
- 4. Camada Convolutacional**
5. Campos Receptivos
6. CNN em 2D
7. Treinamento
8. Downsampling e Upsampling
9. Implementação de Convolução
10. Referências

Camada Convolutiva

- Uma camada convolutiva computa a convolução, soma o *bias* e aplica uma função de ativação:

$$h_i = \varphi[\beta + w_1x_{i-1} + w_2x_i + w_3x_{i+1}] = \varphi\left[\beta + \sum_{j=1}^3 w_jx_{i+j-2}\right]$$

- β , w_1 , w_2 e w_3 são os parâmetros aprendidos.

Camada Convolutiva vs Totalmente Conectada

- A camada convolutiva é um caso especial de camada totalmente conectada.

$$h_i = \varphi \left[\beta + \sum_{j=1}^3 w_j x_{i+j-2} \right]$$

Convolutiva

$$h_i = \varphi \left[\beta_i + \sum_{j=1}^D w_{ij} x_j \right]$$

Totalmente Conectada

- Ou seja, a convolução é uma camada totalmente conectada com pesos compartilhados e a maior parte dos pesos zerados (esparsa e estruturada).

Múltiplos Canais e *Feature Maps*

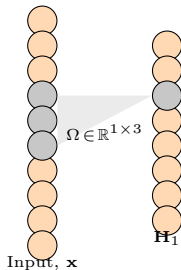
- Tipicamente temos diversos canais de entrada e saída em uma camada convolucional.
- As ativações resultantes de um filtro são armazenadas em unidades ocultas tipicamente denominadas *feature maps* (ou canais).
- A aplicação de filtros diferentes na mesma entrada gera *feature maps* diferentes.
- Em camadas posteriores, cada nova convolução atua sobre todos os *feature maps* simultaneamente.

Overview

1. Motivação
2. Invariância e Equivariância
3. Convolução
4. Camada Convolutiva
- 5. Campos Receptivos**
6. CNN em 2D
7. Treinamento
8. Downsampling e Upsampling
9. Implementação de Convolução
10. Referências

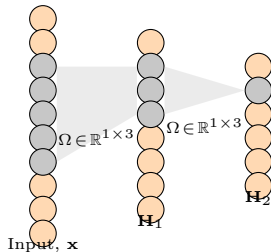
Campo Receptivo (*Receptive Field*)

- Ao realizar uma convolução, cada unidade oculta do *feature map* é gerada levando em conta apenas algumas das entradas.
 - Damos o nome de **Campo Receptivo** à região da entrada considerada por uma unidade oculta.
- Para um *kernel* 1×3 (tamanho 3, 1 canal), cada unidade $\mathbf{H}_1$ enxerga uma janela de tamanho 3 da entrada.



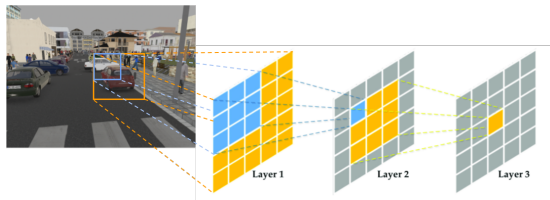
Crescimento do Campo Receptivo

- Empilhando convoluções sucessivas, o campo receptivo cresce camada a camada.
 - Cada unidade de $\mathbf{H}_2$ enxerga 3 unidades de $\mathbf{H}_1$, que por sua vez vêm de uma janela maior na entrada.
 - Isso permite construir representações cada vez mais “globais” da imagem.
- Convoluções com *stride* maior que 1 aceleram esse crescimento, ao custo de reduzir a resolução espacial.



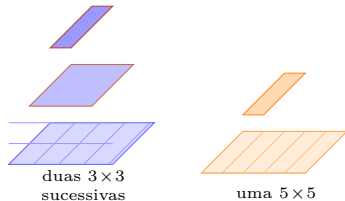
Campos Receptivos (2D)

- Em 2D a ideia se mantém: uma cadeia de convoluções 3×3 produz um campo receptivo crescente em cada camada.
- O campo receptivo de duas convoluções 3×3 sucessivas é igual ao de uma convolução 5×5 .
- Um campo receptivo pequeno pode perder informações relevantes.
- Um campo receptivo muito grande, no limite, equivale a uma camada densa.



Duas 3×3 ou uma 5×5 ?

- Mesmo campo receptivo, diferentes propriedades.
- Resposta: **duas convoluções 3×3 ^a**.
 - Menos parâmetros: $2 \cdot (3 \cdot 3) = 18$ pesos contra $5 \cdot 5 = 25$.
 - Mais não linearidade: duas ativações em vez de uma.



^aKaren Simonyan e Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". Em: [arXiv preprint arXiv:1409.1556](https://arxiv.org/abs/1409.1556) (2014).

Overview

1. Motivação
2. Invariância e Equivariância
3. Convolução
4. Camada Convolutiva
5. Campos Receptivos
- 6. CNN em 2D**
7. Treinamento
8. Downsampling e Upsampling
9. Implementação de Convolução
10. Referências

- Estender o conceito de convolução para 2D é natural.

$$h_{i,j} = \varphi \left[\beta + \sum_{m=1}^3 \sum_{n=1}^3 w_{m,n} x_{i+m-2, j+n-2} \right]$$

- Cada posição (i, j) do *feature map* de saída resulta de uma combinação linear de uma janela 3×3 da entrada, seguida da soma do *bias* e da função de ativação.
- Em imagens RGB, a convolução também é aplicada ao longo dos canais: o *kernel* tem forma $C_i \times K \times K$ por filtro de saída.

Parâmetros Treináveis

- Considerando C_i canais de entrada e C_o canais de saída, com *kernels* $K \times K$:

$$w \in \mathbb{R}^{C_i \times C_o \times K \times K}, \quad \beta \in \mathbb{R}^{C_o}$$

Exemplo

Para 3 canais de entrada, 5 canais de saída e *kernels* 3×3 temos $3 \cdot 5 \cdot 3 \cdot 3 = 135$ pesos nos *kernels*, mais 5 *bias*.

Exemplo de CNN em MNIST 1D³

- Quatro camadas ocultas:
 - 3 convolucionais.
 - 1 totalmente conectada.
 - ~ 2050 parâmetros.
 - 100 000 iterações.
 - SGD, *learning rate* = 0,01.
 - *Batch size* = 100.
- Entrada: $\mathbf{x} \in \mathbb{R}^{40}$.
 - $\mathbf{H}_1 \in \mathbb{R}^{19 \times 15}$, $\mathbf{H}_2 \in \mathbb{R}^{9 \times 15}$, $\mathbf{H}_3 \in \mathbb{R}^{4 \times 15}$.
 - Cada conv com *Size*= 3, *Stride*= 2, 15 canais.
 - Camada final: *Fully Connected* de 60 para 10, seguida de *Softmax*.
 - Saída: $\mathbf{f} \in \mathbb{R}^{10}$.

³Adaptado de Greydanus, S. “Scaling down deep learning”, 2020.

CNN vs MLP em MNIST 1D

- Uma CNN atinge resultados melhores que um MLP no MNIST 1D, com duas ordens de magnitude menos parâmetros treináveis.
 - Convolutacional: ~ 2050 parâmetros, erro de teste estável próximo do erro de treino.
 - Totalmente conectada: $\sim 150,185$ parâmetros, treino vai a zero, mas o erro de teste fica preso em uma plataforma alta.
- Por quê?
 - Pesos compartilhados induzem uma forte regularização estrutural.
 - A inductive bias de localidade espacial é adequada à tarefa.

Overview

1. Motivação
2. Invariância e Equivariância
3. Convolução
4. Camada Convolutiva
5. Campos Receptivos
6. CNN em 2D
- 7. Treinamento**
8. Downsampling e Upsampling
9. Implementação de Convolução
10. Referências

Treinamento

- *Backprop* funciona naturalmente para convoluções.
 - As derivadas parciais com relação a cada peso compartilhado precisam ser **acumuladas** (somadas) sobre todas as posições onde esse peso atua.

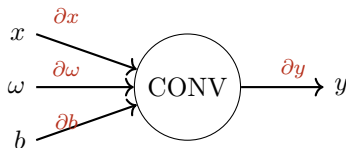
Para um *kernel* 2×2 com pesos $W_{m,n}$ aplicado a uma entrada $X_{i,j}$, a contribuição do gradiente é:

$$\partial W_{1,1} = \sum_{i,j} X_{i,j} \partial h_{i,j}$$

$$\partial W_{1,2} = \sum_{i,j} X_{i,j+1} \partial h_{i,j}$$

$$\partial W_{2,1} = \sum_{i,j} X_{i+1,j} \partial h_{i,j}$$

$$\partial W_{2,2} = \sum_{i,j} X_{i+1,j+1} \partial h_{i,j}$$



O Resultado do Treinamento

- Os *feature maps* das primeiras camadas mostram arestas, cantos e formas primitivas.
- Em camadas mais internas vemos formas mais complexas: olhos, narizes, partes de objetos.
 - Repare que aumenta também o campo receptivo.
- Esse comportamento hierárquico não é programado: emerge naturalmente do treinamento.

Overview

1. Motivação
2. Invariância e Equivariância
3. Convolução
4. Camada Convolutiva
5. Campos Receptivos
6. CNN em 2D
7. Treinamento
- 8. Downsampling e Upsampling**
9. Implementação de Convolução
10. Referências

Downsampling

- Subamostrar espacialmente a imagem pode ter o efeito de aumentar o campo receptivo de uma unidade oculta.
- As principais formas de subamostragem em CNN são:
 1. Usar *stride* maior que 1 na convolução.
 2. *Max pooling*: mantém apenas o maior valor de cada janela.
 3. *Average pooling*: usa a média dos valores de cada janela.

Upsampling

- Sobreamostrar espacialmente pode ser útil quando a saída da rede também é uma imagem (segmentação, super resolução, geração).
- As principais formas de sobreamostragem em CNN são:
 1. Duplicar os valores conhecidos.
 2. *Max unpooling*: insere o valor na posição original do *max pooling* e zera as demais.
 3. Interpolação bilinear.
 4. Convolução transposta.

Convolução Transposta

- Operação aprendível para aumentar a resolução espacial.
- Cada elemento da entrada multiplica o *kernel* inteiro e o resultado é depositado em uma região da saída, com sobreposições somadas.
- Parâmetros: *kernel size*, *padding*, *stride*. *Stride* maior que 1 **expande** a saída.

Convolução Transposta: Exemplo

- Filtro 3×3 , $padding = 1$, $stride = 2$.

feature map de entrada

$$\begin{pmatrix} 2 & 1 \\ 3 & 2 \end{pmatrix}$$

kernel

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \\ 0 & 2 & 1 \end{pmatrix}$$

resultado 4×4

$$\begin{pmatrix} 0 & 4 & 0 & 1 \\ 10 & 7 & 6 & 3 \\ 0 & 7 & 0 & 2 \\ 6 & 3 & 4 & 2 \end{pmatrix}$$

- Cada entrada projeta o *kernel* inteiro deslocado de *stride* posições. As regiões sobrepostas são somadas para formar a saída final.

Convolução 1×1

- Algumas vezes queremos mudar o número de canais sem agrupamento espacial.
 - Nesses casos podemos aplicar a convolução 1×1 .
- Cada posição espacial é independente: cada saída $h'_{i,j,c}$ é uma combinação linear dos canais de entrada $h_{i,j,1...C_i}$.
- Útil para reduzir ou expandir a profundidade do tensor sem alterar largura e altura.

Overview

1. Motivação
2. Invariância e Equivariância
3. Convolução
4. Camada Convolutiva
5. Campos Receptivos
6. CNN em 2D
7. Treinamento
8. Downsampling e Upsampling
- 9. Implementação de Convolução**
10. Referências

Implementação: Versão Ingênua

- Itera explicitamente por linhas, colunas, canais e *kernel*:

```
1 for m in range(0, M):           # linhas
2     for n in range(0, N):       # colunas
3         for c in range(0, C):   # canais
4             for i in range(0, K): # tamanho kernel
5                 for j in range(0, K): # tamanho kernel
6                     feat_map_out[m, n] += kernel[i, j, c] * feat_map_in[m+i, n+j, c]
7                 feat_map_out[m, n] += bias
```

- Simples de entender, mas extremamente lenta e sem aproveitar paralelismo de hardware.

Implementação: via Transformada de Fourier

- Versão complexa e raramente utilizada na prática.
- Convolução no domínio espacial é equivalente a multiplicação no domínio da frequência:

$$g(x) = f(x) * h(x) \quad \Longleftrightarrow \quad G(u) = F(u) H(u)$$

- Fluxo: $f(x), h(x) \rightarrow FFT \rightarrow F(u), H(u) \rightarrow$ multiplicação ponto a ponto $\rightarrow IFFT \rightarrow f(x) * h(x)$.
- Útil para convoluções com *kernels* grandes (acima de 11×11)⁴.

⁴Nicolas Vasilache et al. “Fast convolutional nets with fbfft: A GPU performance evaluation”. [Em: arXiv preprint arXiv:1412.7580](#) (2014).

Implementação: im2col e Matriz de Toeplitz

- Versão utilizada na prática em GPUs e bibliotecas otimizadas.
- Existe uma forma inteligente de organizar a matriz de pesos para obter o resultado da convolução com uma única multiplicação de matrizes.
- Uma **matriz de Toeplitz** é uma matriz cujas diagonais principal e subdiagonais são constantes:

$$\begin{bmatrix} 5 & 27 & 2 & -56 & 0 & 1 \\ -3 & 5 & 27 & 2 & -56 & 0 \\ 8 & -3 & 5 & 27 & 2 & -56 \\ 1 & 8 & -3 & 5 & 27 & 2 \\ -9 & 1 & 8 & -3 & 5 & 27 \\ 3 & -9 & 1 & 8 & -3 & 5 \end{bmatrix}$$

im2col: Roteiro

1. Definir a entrada X e o filtro w .
2. Calcular o tamanho da saída: $(X_h + w_h - 1) \times (X_w + w_w - 1)$. Para X 3×3 e w 2×2 , saída 4×4 .
3. Aplicar *zero padding* no filtro até atingir o tamanho da saída.
4. Construir uma matriz de Toeplitz F_k para cada linha do filtro com *padding*, começando da última linha.
5. Empilhar as F_k em uma matriz de **bloco Toeplitz** (cada bloco repete o padrão da convolução).
6. Vetorizar a entrada X concatenando as linhas de baixo para cima.
7. Multiplicar a matriz de bloco Toeplitz pela entrada vetorizada.
8. Fazer o *reshape* do resultado para a forma da saída.

im2col: Estrutura da Matriz de Bloco Toeplitz

- Após o roteiro, a matriz final tem a forma:

$$\text{Bloco Toeplitz} = \begin{bmatrix} F_0 & 0 & 0 \\ F_1 & F_0 & 0 \\ F_2 & F_1 & F_0 \\ F_3 & F_2 & F_1 \end{bmatrix}$$

- O número de *colunas* corresponde ao número de *linhas* da imagem original.
- O número de *linhas* corresponde ao número de *linhas* da imagem de saída.
- Multiplicando essa matriz pela imagem vetorizada obtemos um vetor que, após *reshape*, é exatamente o resultado da convolução tradicional.

Por que `im2col` é a escolha prática

- Reduz a convolução a uma multiplicação de matrizes (GEMM), o tipo de operação que GPUs e bibliotecas BLAS otimizam ao máximo.
- Permite usar a mesma rotina para qualquer combinação de *kernel*, *stride* e *padding*, alterando apenas o layout dos dados.
- Custa memória extra para armazenar as “cópias” das janelas, mas o ganho em velocidade compensa em larga escala.

Overview

1. Motivação
2. Invariância e Equivariância
3. Convolução
4. Camada Convolutacional
5. Campos Receptivos
6. CNN em 2D
7. Treinamento
8. Downsampling e Upsampling
9. Implementação de Convolução
- 10. Referências**

Referências I

- Sugere-se **fortemente** a leitura do Capítulo 10 de *Understanding Deep Learning*⁵.
 - Disponível em <https://udlbook.github.io/udlbook/>.
- Aula baseada também nos seguintes materiais:
 - Capítulo 10 de *Understanding Deep Learning*, Simon J. D. Prince.
 - *Machine Learning* @ Berkeley.
 - https://github.com/alisaaalehi/convolution_as_multiplication.
 - https://github.com/vdumoulin/conv_arithmetic.

- [1] David H. Hubel e Torsten N. Wiesel. “Receptive fields, binocular interaction and functional architecture in the cat’s visual cortex”. Em: [The Journal of Physiology](#) 160.1 (1962), pp. 106–154.
- [2] Alex Krizhevsky, Ilya Sutskever e Geoffrey E. Hinton. “ImageNet classification with deep convolutional neural networks”. Em: [Advances in Neural Information Processing Systems](#). Vol. 25. 2012.
- [3] Yann LeCun et al. “Handwritten digit recognition with a back-propagation network”. Em: [Advances in Neural Information Processing Systems](#) 2 (1989).
- [4] Simon J. D. Prince. [Understanding Deep Learning](#). MIT Press, 2023.
- [5] Karen Simonyan e Andrew Zisserman. “Very deep convolutional networks for large-scale image recognition”. Em: [arXiv preprint arXiv:1409.1556](#) (2014).
- [6] Nicolas Vasilache et al. “Fast convolutional nets with fbfft: A GPU performance evaluation”. Em: [arXiv preprint arXiv:1412.7580](#) (2014).

⁵Simon J. D. Prince. [Understanding Deep Learning](#). MIT Press, 2023.